CSE 385: Analysis of Algorithms (Honors)

Lecture 21
(Reductions and NP-Completeness)

Rezaul A. Chowdhury

Department of Computer Science
SUNY Stony Brook
Spring 2026

Optimization Problems vs Decision Problems

Optimization Problem:

An *optimization problem* is one in which each feasible (i.e., "legal") solution has an associated value, and we wish to find a feasible solution with the best value.

For example, in a problem that we call SHORTEST-PATH we are given a directed graph G and vertices u and v, and we wish to find a path from u to v that uses the fewest edges.

Decision Problem:

A *decision problem* is one in which the answer (i.e., solution) is simply Y/Yes (1) or N/No (0).

For example, in a problem that we call PATH we are given a directed graph G, vertices u and v, and an integer k, and we ask: does a path exist from u to v consisting of at most k edges?

P, NP, NP-Complete, NP-Hard

Complexity Class P (Polynomial **(**time**)):**

The class P is the set of decision problems that are solvable in polynomial time.

More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k, where n is the size of the input to the problem.

P, NP, NP-Complete, NP-Hard

Complexity Class NP (Nondeterministic **P**olynomial **(**time**)):**

The class NP is the set of decision problems with the following property: If the answer is Y, then there is a proof of this fact that can be checked in polynomial time.

Intuitively, NP is the set of decision problems where we can verify a Y answer quickly if we have the solution in front of us.

For example, in the HAMILTONIAN-CYCLE problem, given a directed graph $G = (V, E)$, a solution would be a sequence $\langle v_1, v_2, \dots, v_{|V|} \rangle$ of $|V|$ vertices. We could easily check in polynomial time that $(v_i, v_{i+1}) \in E$ for $i = 1,2,3, \dots, |V| - 1$ and that $(v_{|V|}, v_1) \in E$ as well.

Note that P $\subseteq$ NP.

P, NP, NP-Complete, NP-Hard

NP-Complete Problems:

A problem is NP-complete if it is in NP and is as “hard” as any problem in NP.

If any NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomial-time algorithm.

Hard problems (**NP**-complete)	Easy problems (in **P**)
3SAT	2SAT, HORN SAT
TRAVELING SALESMAN PROBLEM	MINIMUM SPANNING TREE
LONGEST PATH	SHORTEST PATH
3D MATCHING	BIPARTITE MATCHING
KNAPSACK	UNARY KNAPSACK
INDEPENDENT SET	INDEPENDENT SET on trees
INTEGER LINEAR PROGRAMMING	LINEAR PROGRAMMING
RUDRATA PATH	EULER PATH
BALANCED CUT	MINIMUM CUT

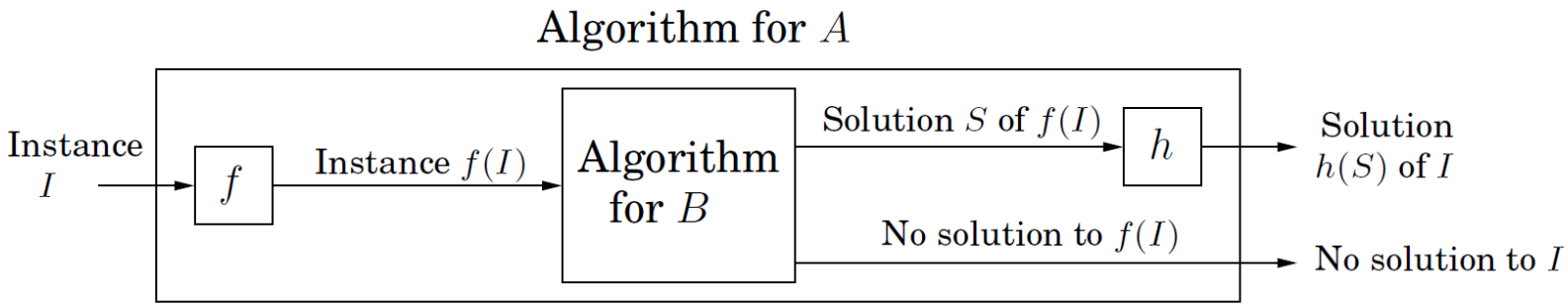
P, NP, NP-Complete, NP-Hard

NP-Hard Problems:

A problem is NP-hard if it is as “hard” as any problem in NP but may or may not belong to NP.

For example, optimization versions of NP-Complete decision problems are NP-Hard.

Reductions

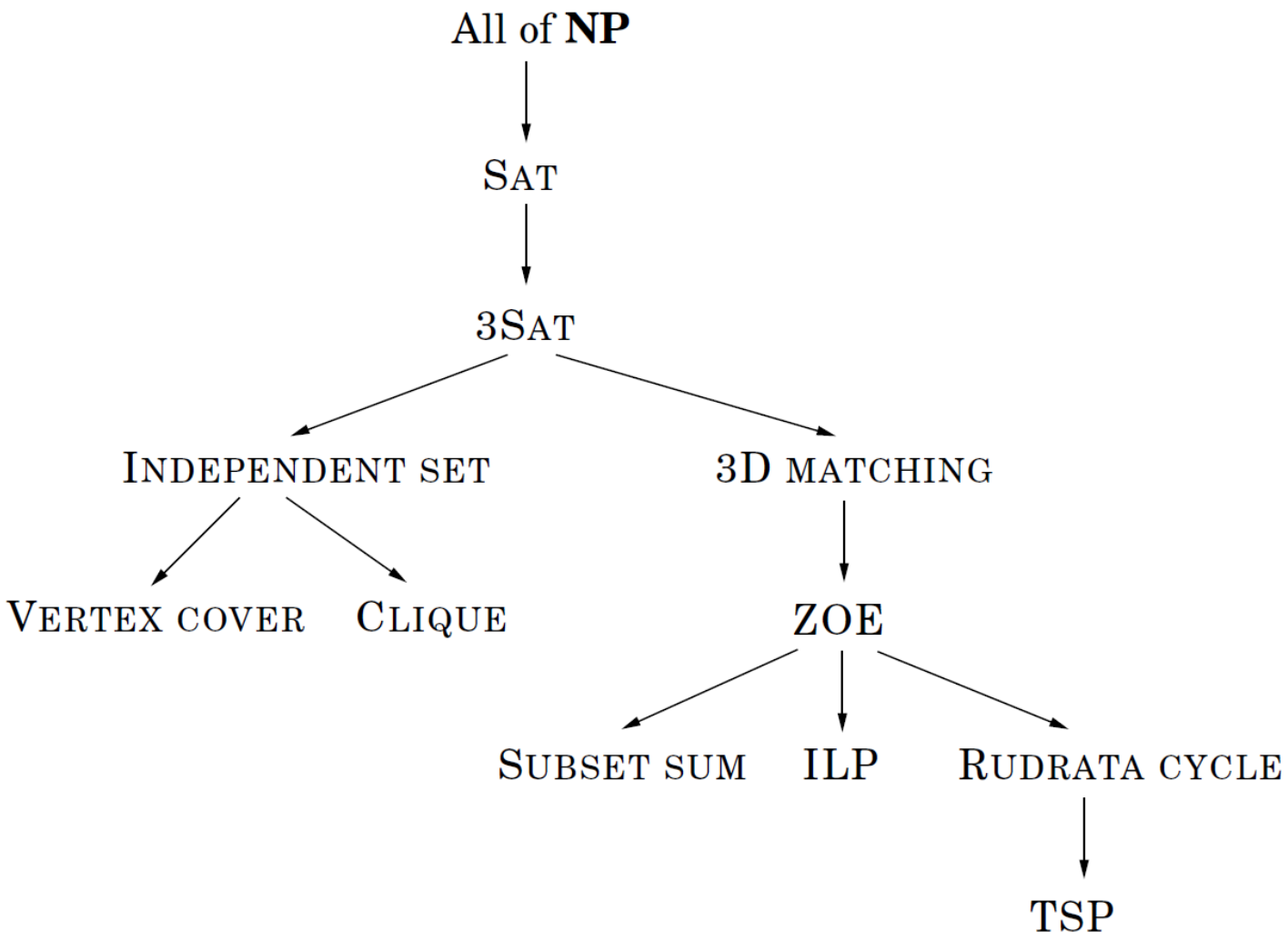


A reduction from a problem A to problem B is a polynomial-time algorithm f that transforms any instance I of A into an instance $f(I)$ of B, together with another polynomial-time algorithm h that maps any solution S of $f(I)$ back into a solution $h(S)$ of I.

If $f(I)$ has no solution, then neither does I.

A decision problem is NP-Complete if all other problems in NP can be reduced to it.

Reductions


All of NP
SAT
3SAT
INDEPENDENT SET
3D MATCHING
VERTEX COVER
CLIQUE
ZOE
SUBSET SUM
ILP
RUDRATA CYCLE
TSP

Decision Problem: Satisfiability (SAT/CNFSAT)

The SAT (sometimes called CNFSAT) problem is the following: given a Boolean formula in conjunctive normal form, either find a satisfying truth assignment or else report that none exists.

$$(x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (z \vee \bar{x}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

This is a Boolean formula in conjunctive normal form (CNF). It is a collection of clauses (the parentheses), each consisting of the disjunction (logical or, denoted $\vee$) of several literals, where a literal is either a Boolean variable (such as x) or the negation of one (such as $\bar{x}$).

A satisfying *truth* assignment is an assignment of *false* or *true* to each variable so that every clause contains at least one literal whose value is *true*.

Decision Problem: k-SATISFIABILITY (k-SAT)

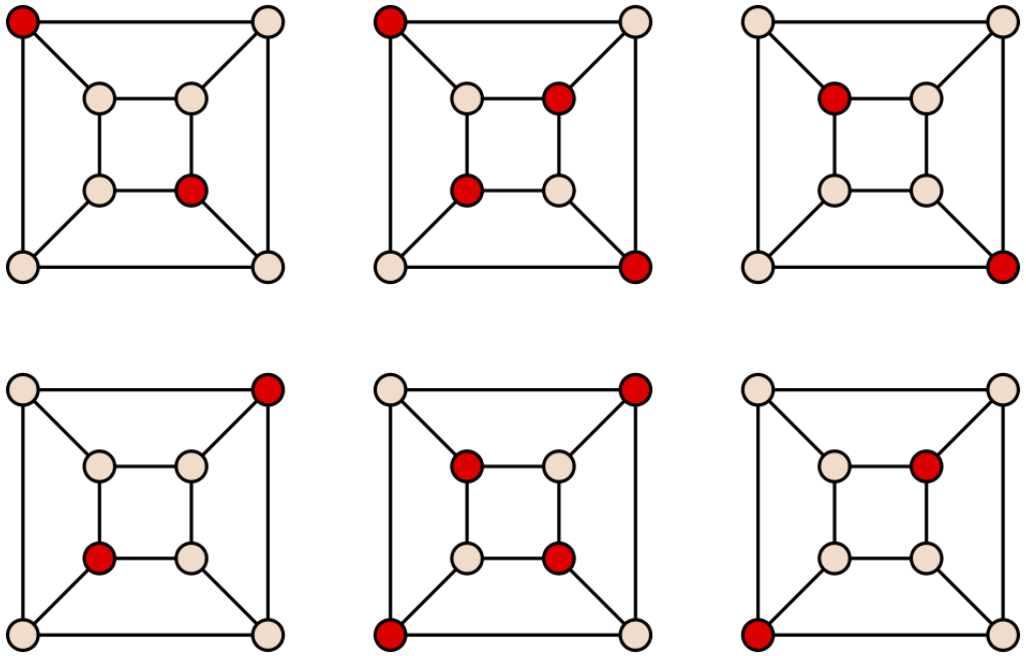
The k-SAT (sometimes called k-CNFSAT) problem is a restricted version of the SAT problem, where each clause in the given Boolean formula is restricted to have at most k literals.

For example, the following has at most 3 literals per clause:

$$(x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (z \vee \bar{x}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

Decision Problem: INDEPENDENT SET

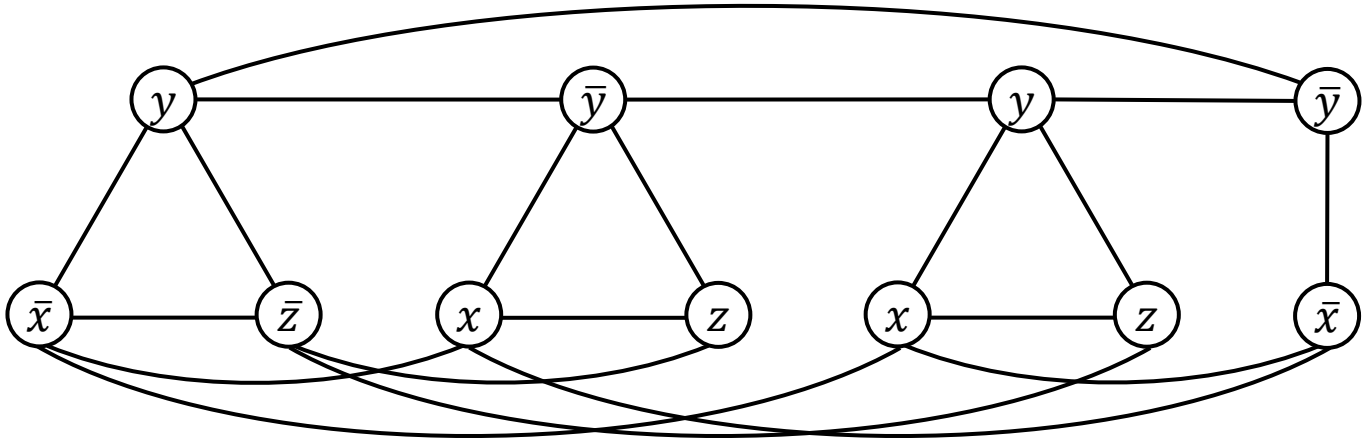
In the INDEPENDENT SET problem, we are given a graph and an integer k, and the aim is to find k vertices that are independent, that is, no two of which have an edge between them.



Some Independent Sets (red vertices) of the Cube Graph
Source: Wikipedia

Reduction: 3-SAT → Independent Set

$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$

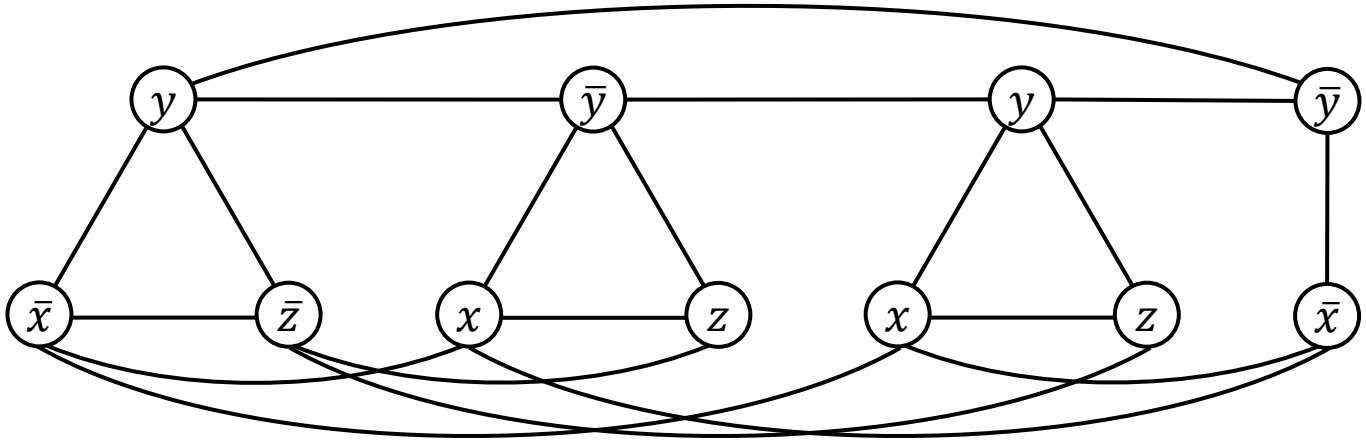


Given an instance I of 3-SAT, we create an instance (G, k) of Independent Set as follows.

- Graph G has a triangle for each clause (just an edge for a clause with two literals), with vertices labeled by the clause's literals.
- Add extra edges between vertices that represent opposite literals.
- The goal k is set to the number of clauses.

Reduction: 3-SAT → Independent Set

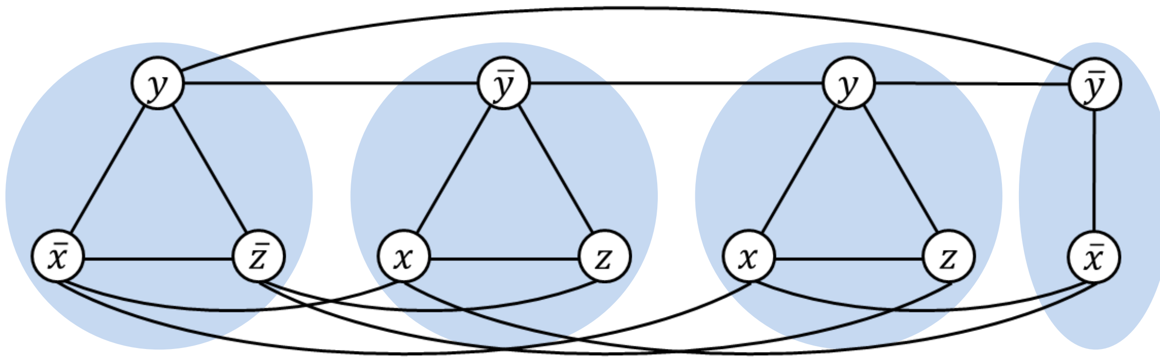
$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$



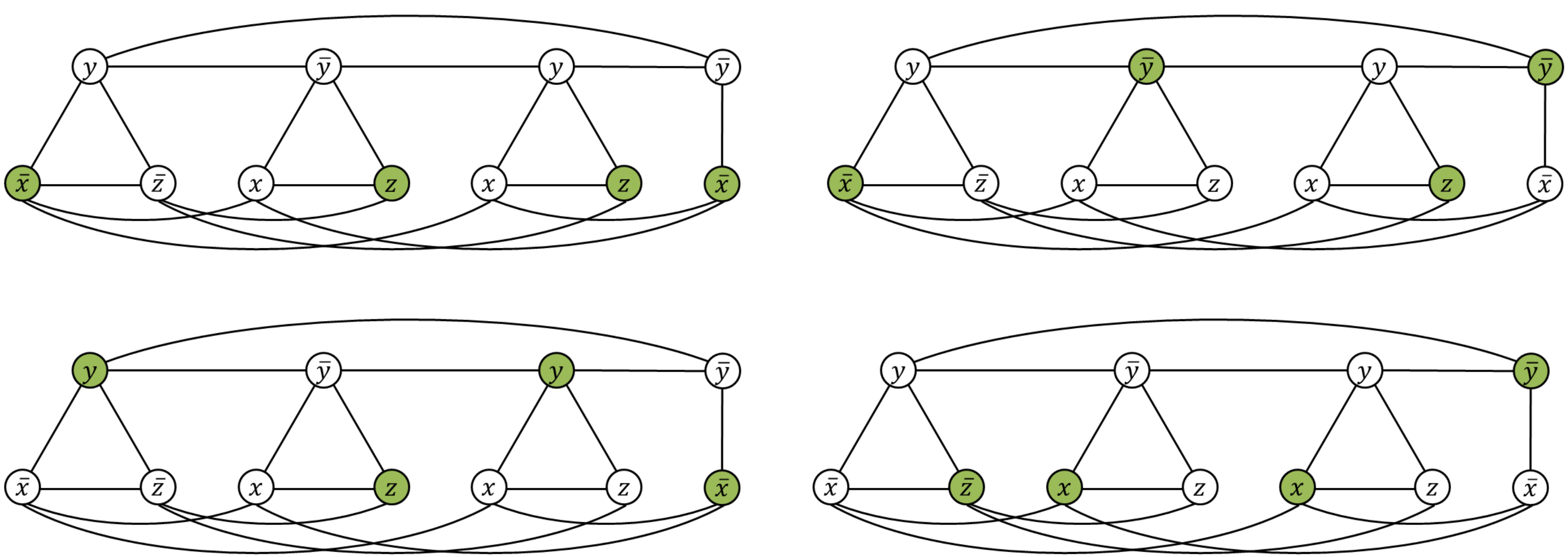
- Given an independent set S of k vertices in G, it is possible to efficiently recover a satisfying truth assignment to I.
- If graph G has no independent set of size k then Boolean formula I is unsatisfiable.

Reduction: 3-SAT → INDEPENDENT SET

$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$

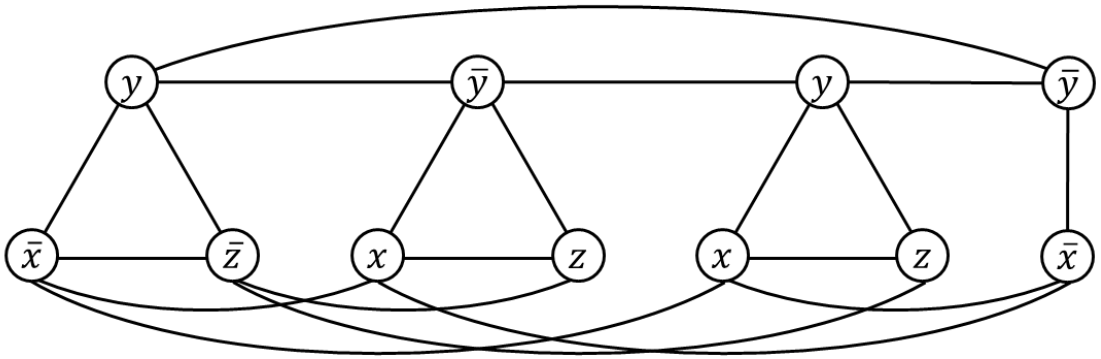


Few Independent Sets of Size 4



Reduction: 3-SAT → Independent Set

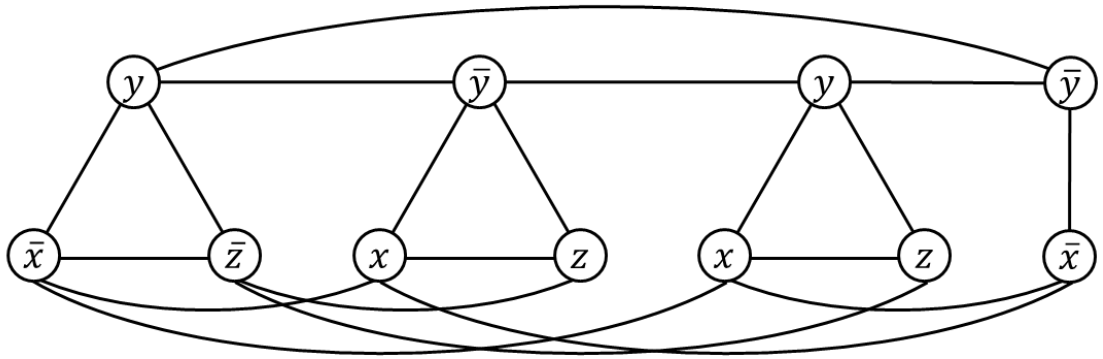
$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$



- Given an independent set S of k vertices in G, it is possible to efficiently recover a satisfying truth assignment to I.

Reduction: 3-SAT → Independent Set

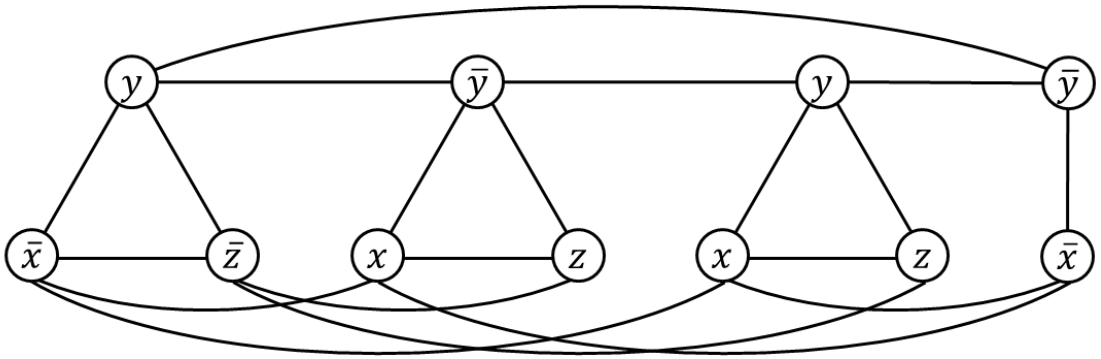
$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$



- Given an independent set S of k vertices in G, it is possible to efficiently recover a satisfying truth assignment to I.

Reduction: 3-SAT → Independent Set

$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$



- Given an independent set S of k vertices in G, it is possible to efficiently recover a satisfying truth assignment to I.

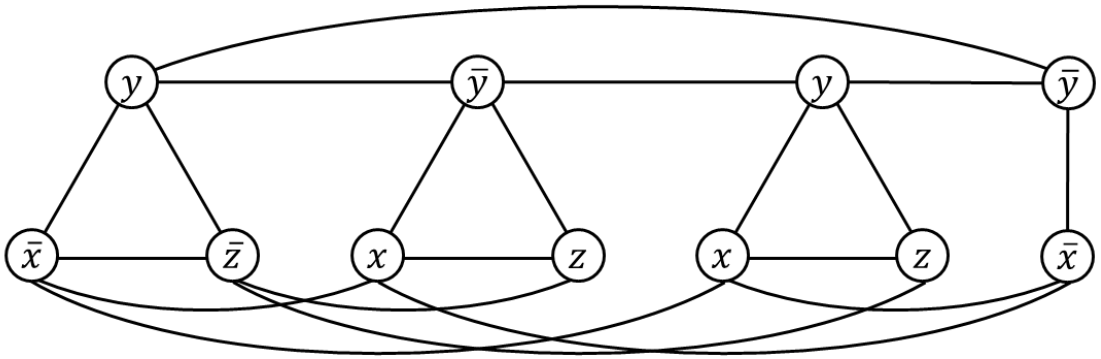
For any variable x, the set S cannot contain vertices labeled both x and $\bar{x}$, because any such pair of vertices is connected by an edge.

So, assign x a value of TRUE if S contains a vertex labeled x, and a value of FALSE if S contains a vertex labeled $\bar{x}$ (if S contains neither, then assign either value to x).

Since S has k vertices, it must have one vertex per clause; this truth assignment satisfies those particular literals, and thus satisfies all clauses.

Reduction: 3-SAT → Independent Set

$$I = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$



- If graph G has no independent set of size k then Boolean formula I is unsatisfiable.

It is usually cleaner to prove the contrapositive, that if I has a satisfying assignment then G has an independent set of size k.

This is easy: for each clause, pick any literal whose value under the satisfying assignment is TRUE (there must be at least one such literal), and add the corresponding vertex to S.

Then the set S must be independent. (why?)

Reduction: SAT → 3-SAT

Given an instance I of SAT, use exactly the same instance for 3-SAT, except that any clause with more than three literals,

$$(a_1 \vee a_2 \vee \cdots \vee a_k) \text{ (where the } a_i\text{'s are literals and } k > 3),$$

is replaced by a set of clauses,

$$(a_1 \vee a_2 \vee y_1) \wedge (\bar{y}_1 \vee a_3 \vee y_2) \wedge (\bar{y}_2 \vee a_4 \vee y_3) \wedge \cdots \wedge (\bar{y}_{k-3} \vee a_{k-1} \vee a_k),$$

where the y_i's are new variables.

Call the resulting 3-SAT instance I'.

The conversion from I to I' is clearly polynomial time.

I' is equivalent to I in terms of satisfiability, because for any assignment to the a_i's,

$$\left\{\begin{array}{c}(a_1 \vee a_2 \vee \cdots \vee a_k)\\ \text{is satisfied}\end{array}\right\} \Leftrightarrow \left\{\begin{array}{c}\text{there is a setting of the } y_i\text{'s for which}\\ (a_1 \vee a_2 \vee y_1) \wedge (\bar{y}_1 \vee a_3 \vee y_2) \wedge \cdots \wedge (\bar{y}_{k-3} \vee a_{k-1} \vee a_k)\\ \text{are all satisfied}\end{array}\right\}$$

Reduction: SAT → 3-SAT

I' is equivalent to I in terms of satisfiability, because for any assignment to the a_i's,

$$\left\{\begin{array}{c}(a_1 \vee a_2 \vee \cdots \vee a_k)\\ is\ satisfied\end{array}\right\} \Leftrightarrow \left\{\begin{array}{c}there\ is\ a\ setting\ of\ the\ y_i's\ for\ which\\ (a_1 \vee a_2 \vee y_1) \wedge (\bar{y}_1 \vee a_3 \vee y_2) \wedge \cdots \wedge (\bar{y}_{k-3} \vee a_{k-1} \vee a_k)\\ are\ all\ satisfied\end{array}\right\}$$

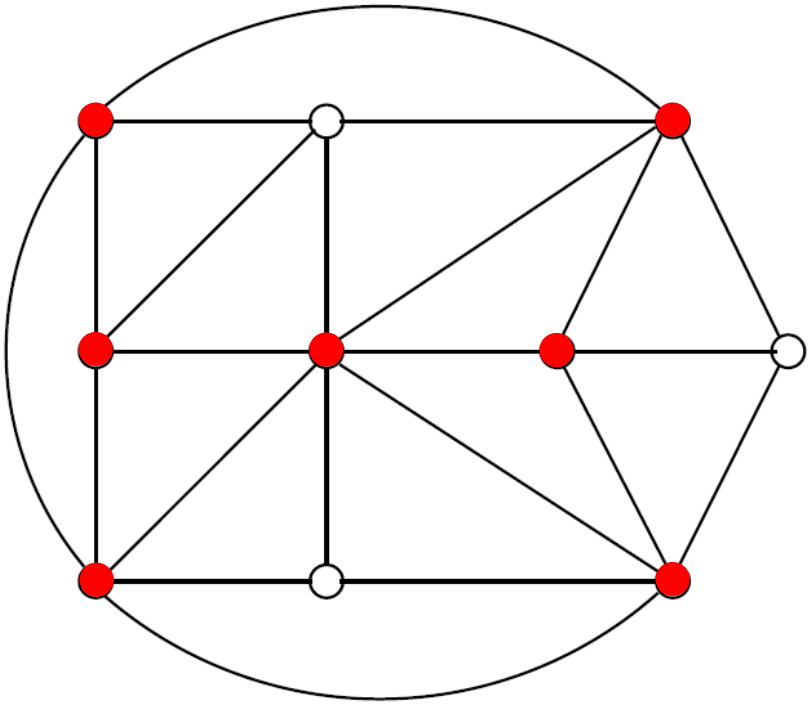
To see this, first suppose that the clauses on the right are all satisfied. Then at least one of the literals $a_1, a_2, \ldots, a_k$ must be TRUE as otherwise y_1 would have to be TRUE, which would in turn force y_2 to be true, and so on, eventually falsifying the last clause. But this means $(a_1 \vee a_2 \vee \cdots \vee a_k)$ is also satisfied.

Conversely, if $(a_1 \vee a_2 \vee \cdots \vee a_k)$ is satisfied, then some a_i must be TRUE. Set $y_1, y_2, \ldots, y_{i-2}$ to TRUE and the rest to FALSE. This ensures that the clauses on the right are all satisfied.

Decision Problem: VERTEX COVER

In the VERTEX COVER problem, we are given a graph G and a budget k, and we are asked if there exists a set of k vertices that cover (touch) every edge.

All edges of the following graph can be covered with 7 vertices.

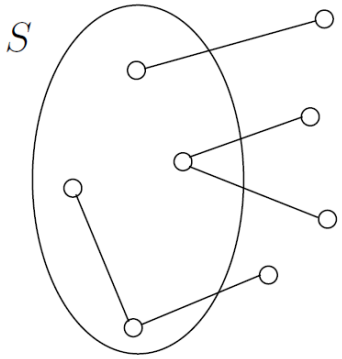


Can they be covered using only 6 vertices?

Reduction: Independent Set → Vertex Cover

To reduce Independent Set to Vertex Cover we just need to observe that a set of nodes S is a vertex cover of graph $G = (V, E)$ (that is, S touches every edge in E) if and only if the remaining nodes, $V \setminus S$ are an independent set of G.

S is a vertex cover if and only if $V - S$ is an independent set.

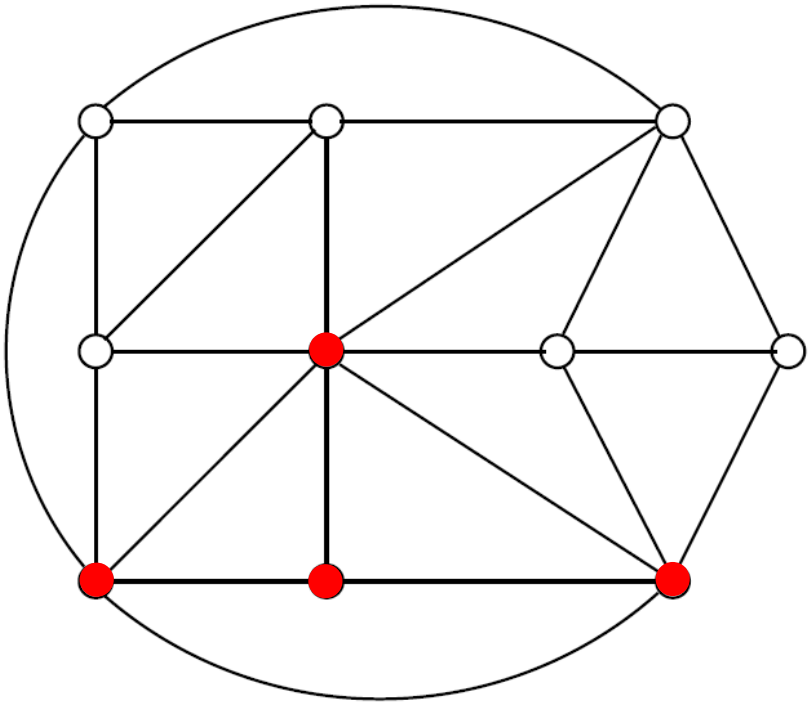


Therefore, to solve an instance (G, k) of Independent Set, simply look for a vertex cover of G with $|V| - k$ nodes. If such a vertex cover exists, then take all nodes not in it. If no such vertex cover exists, then G cannot possibly have an independent set of size k.

Decision Problem: CLIQUE

In the CLIQUE problem, we are given a graph G and a positive integer k, and we are asked if there exists a set of k vertices such that all edges between them are present.

There is a clique of size 4 in the following graph.



What is the size of the largest clique in the graph above?

Reduction: Independent Set → Clique

Independent Set and Clique easily reduce to one another.

Define the complement of a graph $G = (V, E)$ to be $G' = (V, E')$, where E' contains precisely those unordered pairs of vertices that are not in E. Then a set of nodes S is an independent set of G if and only if S is a clique of G'.

To paraphrase, these nodes in S have no edges between them in G if and only if they have all possible edges between them in G'.

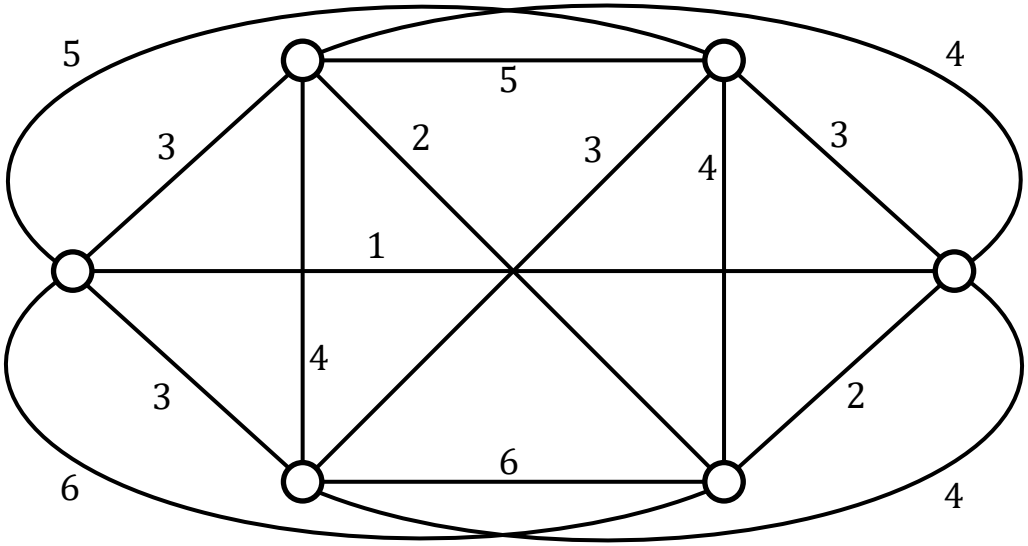
Therefore, we can reduce Independent Set and Clique by mapping an instance (G, k) of Independent Set to the corresponding instance (G', k) of Clique; the solution to both is identical.

Decision Problem: Traveling Salesman Problem (TSP)

In the Traveling Salesman Problem (TSP) we are given n vertices $1,2,\dots,n$ and all $n(n-1)/2$ distances between them, as well as a budget b. We are asked to find a *tour*, a cycle that passes through every vertex exactly once, of total cost b or less, or to report that no such tour exists.

We seek a permutation $\tau(1),\tau(2),\dots,\tau(n)$ of the vertices such that when they are toured in this order, the total distance covered $\leq b$:

$$d_{\tau(1),\tau(2)} + d_{\tau(2),\tau(3)} + \cdots + d_{\tau(n-1),\tau(n)} + d_{\tau(n),\tau(1)} \leq b.$$

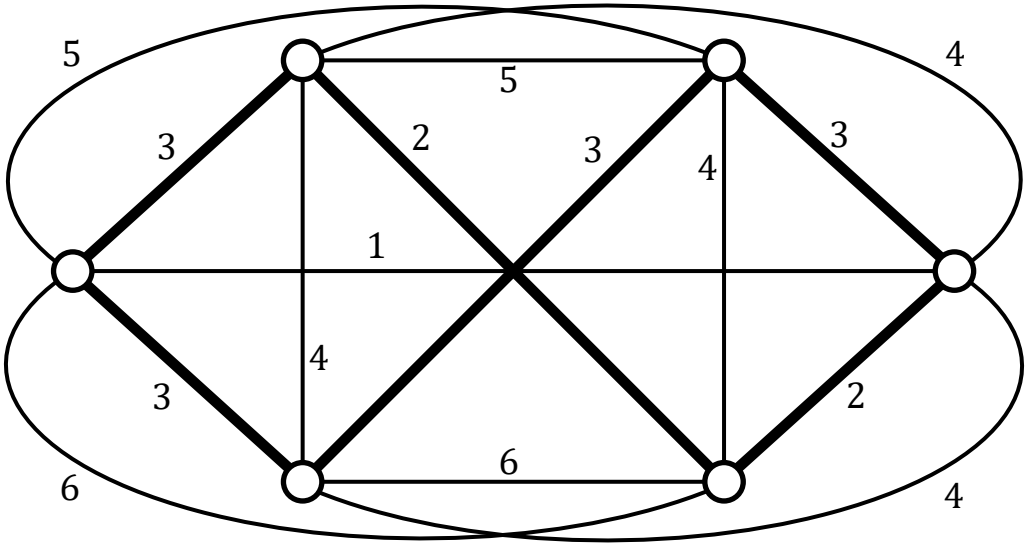


Decision Problem: Traveling Salesman Problem (TSP)

In the Traveling Salesman Problem (TSP) we are given n vertices $1,2,\ldots,n$ and all $n(n-1)/2$ distances between them, as well as a budget b. We are asked to find a *tour*, a cycle that passes through every vertex exactly once, of total cost b or less, or to report that no such tour exists.

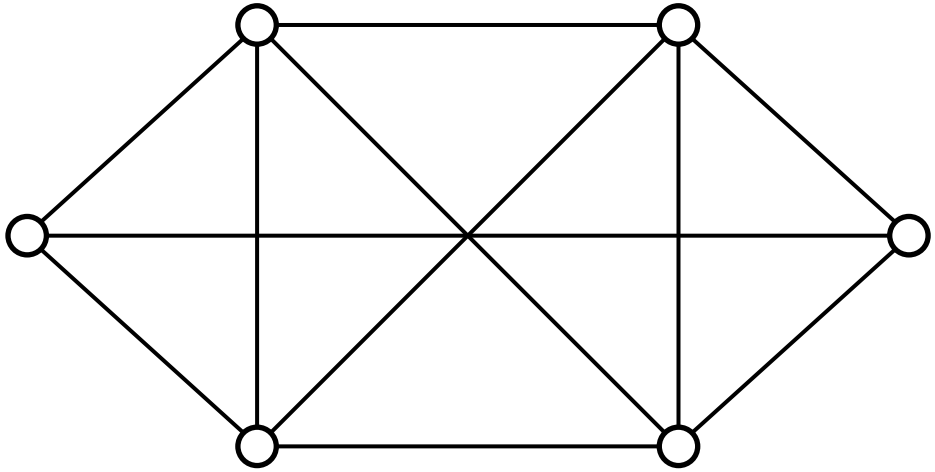
We seek a permutation $\tau(1),\tau(2),\ldots,\tau(n)$ of the vertices such that when they are toured in this order, the total distance covered $\leq b$:

$$d_{\tau(1),\tau(2)} + d_{\tau(2),\tau(3)} + \cdots + d_{\tau(n-1),\tau(n)} + d_{\tau(n),\tau(1)} \leq b.$$



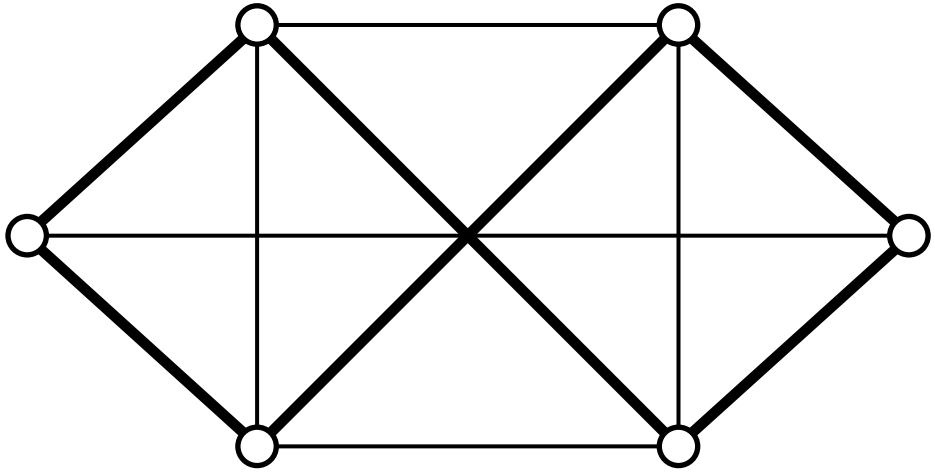
Decision Problem: HAMILTONIAN CYCLE

In the HAMILTONIAN CYCLE problem, we are given a graph $G = (V, E)$, and we are asked to a find a cycle that visits each vertex in V exactly once, or report that no such cycle exists.



Decision Problem: HAMILTONIAN CYCLE

In the HAMILTONIAN CYCLE problem, we are given a graph $G = (V, E)$, and we are asked to a find a cycle that visits each vertex in V exactly once, or report that no such cycle exists.

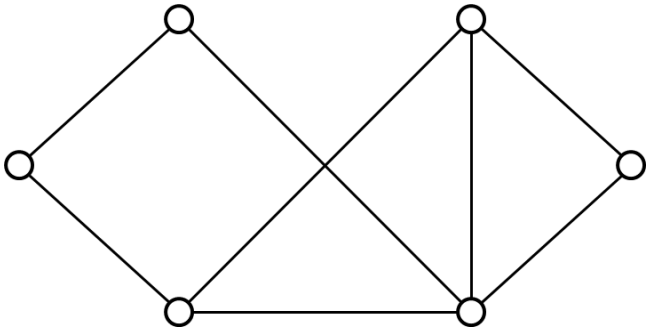


Reduction: Hamiltonian Cycle → TSP

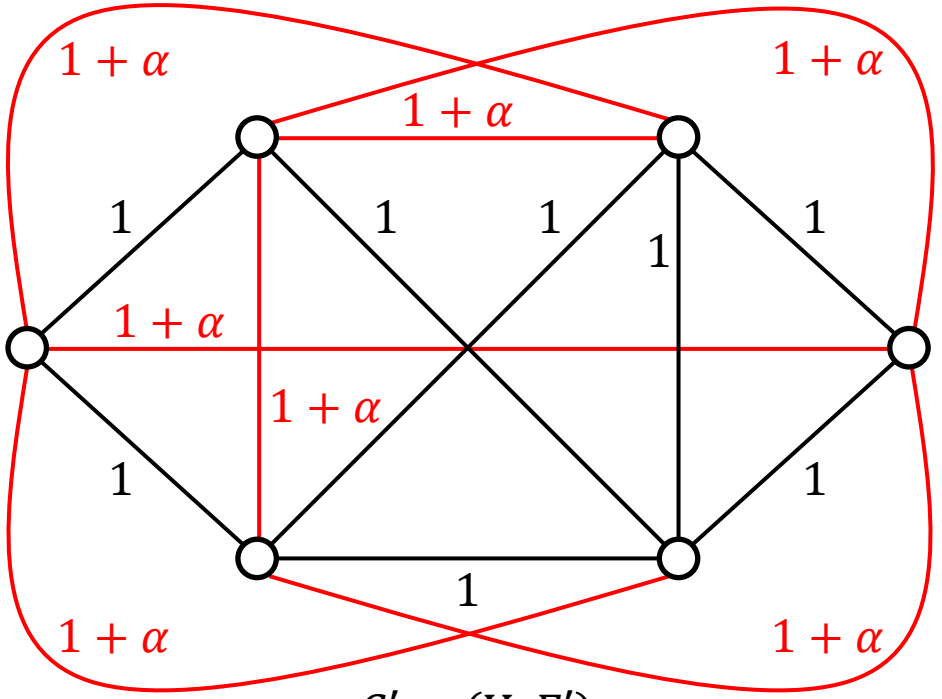
Given a graph $G = (V, E)$ in which we are required to find a Hamiltonian cycle, we construct the following instance of the TSP:

The set of cities is the same as V, and the distance between cities u and v is 1 if $(u, v) \in E$ and $1 + \alpha$ otherwise, for some $\alpha > 1$ to be determined.

The budget of the TSP instance is equal to $|V|$.

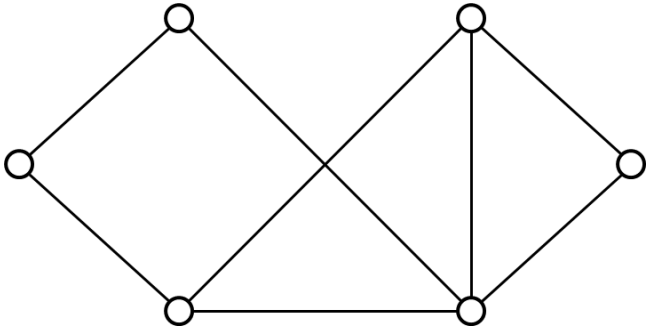


$G = (V, E)$

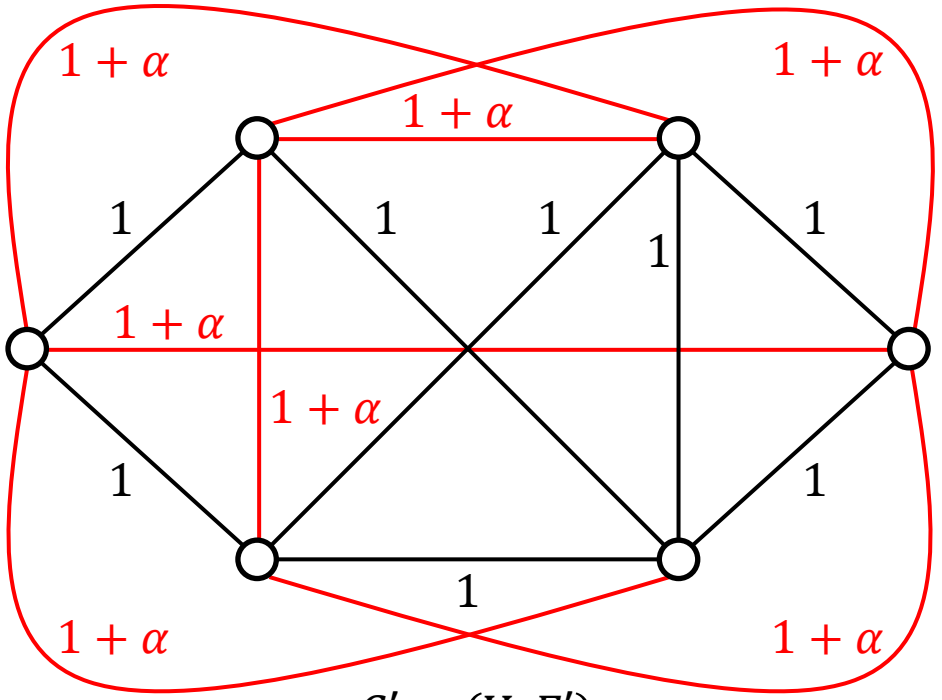


$G' = (V, E')$

Reduction: HAMILTONIAN CYCLE → TSP



$G = (V, E)$



$G' = (V, E')$

If G has a Hamiltonian cycle, then the same cycle is also a tour in G' within the budget of the TSP instance.

If G has no Hamiltonian cycle, then there is no solution: the cheapest TSP tour has cost at least $n + \alpha$ (at least one edge of length $1 + \alpha$, and total length of all $n - 1$ others $\geq n - 1$).

Thus, HAMILTONIAN CYCLE reduces to TSP.